

중간고사 준비

▼ 목차

데이터사이언스기초 (Introduction to Data Science)

Sooyeon Ji, Assistant Professor

Division of Computer Engineering, Hankuk University of Foreign Studies

1. OT & Course Introduction

- 데이터 사이언스 개념 소개
 - 데이터 사이언스의 구성요소 (통계학, 컴퓨터 과학, 도메인 지식)
 - 데이터의 중요성 및 디지털 데이터 개념
 - 데이터 사이언스 사례 소개
 - 존 스노우 콜레라 연구
 - 블랙홀 이미지 생성 (EHT 프로젝트)
 - 디지털 데이터 및 디지털 전환
 - 데이터 중심의 과학 패러다임 변화
 - 강의 학습 목표 및 학습할 내용 개요
-

2. 데이터 분석 도구 및 라이브러리 소개

2.1 Python 개요

- Python의 특징 및 기본 자료구조
- 인덱싱과 슬라이싱
- 객체 지향 프로그래밍 개념

2.2 모듈 및 패키지 활용

- 모듈 개념 및 사용법
- 내장 모듈, 사용자 정의 모듈, 외부 패키지 관리 (pip)

2.3 데이터 사이언스 필수 라이브러리

- NumPy, Pandas
- 머신러닝 라이브러리 (Scikit-learn, TensorFlow)
- 데이터 시각화 라이브러리 (Matplotlib, Seaborn)

2.4 개발 환경 소개

- Anaconda 설치 및 환경 설정
- Jupyter Notebook 개념과 사용법

- Google Colab 소개 및 활용 방법
-

3. NumPy 라이브러리 기초

- NumPy vs Python List 비교
 - 다차원 배열 및 주요 속성(shape, dtype, size, ndim)
 - 브로드캐스팅 및 벡터화 개념
 - 특수 배열 생성법 (zeros, ones, eye 등)
 - 연속된 값 생성 (arange, linspace, logspace)
 - 배열 인덱싱, 슬라이싱, 논리적 인덱싱
 - 기본 통계 및 정렬 연산 (max, min, mean, sort)
 - 배열 결합 및 형태 변환 (append, reshape, concatenate, vstack, hstack)
 - 난수 생성 및 재현성(seed 설정)
 - NumPy 성능 평가 및 벡터화 중요성
 - 선형대수 연산 지원
-

4. 최적화 문제 (Optimization Problems)

- 최적화 모델 개념 및 사례 (배낭 문제: Knapsack Problem)
 - 최적화 문제 유형 (연속적 문제 vs 이산적 문제)
 - 완전 탐색 (Brute Force) 알고리즘의 개념 및 한계
 - 탐욕 알고리즘 (Greedy Algorithm)의 개념 및 장단점
 - 이진 탐색 트리(Binary Search Tree)를 통한 완전 탐색 구현
 - 시간 복잡도 분석
 - 동적 프로그래밍(Dynamic Programming)의 개념과 활용 사례
 - 최적 부분 구조와 중복 부분 문제
 - 최적화 문제 해결을 위한 다양한 알고리즘 성능 비교
-

5. 데이터 시각화 개요

- 시각화의 역할과 중요성
- 기본 시각화 기법 소개:
 - 선 그래프(Line Chart)
 - 파이차트(Pie Chart), 막대그래프(Bar Chart)
 - 히트맵(Heatmap)
 - 산점도(Scatter Plot)
 - 히스토그램(Histogram)
 - 상자그림(Box-and-whisker Plot)
- 시각화 유형 선택 기준 및 예시 문제
- Matplotlib 활용 기초 및 고급 (plt.plot, 서브플롯, Figure와 Axes 활용)

- Iris 데이터셋을 통한 실습 예제 (히스토그램, 산점도, 상자그림)
-

6. Pandas 라이브러리 활용

- Pandas 소개 (Series, DataFrame 구조)
 - NumPy와 비교 및 활용 이점
 - DataFrame 생성 방법 (딕셔너리, CSV, Excel 파일 등)
 - 데이터 결측치 처리 (dropna, fillna 등)
 - 슬라이싱과 인덱싱 (loc, iloc, Boolean 인덱싱)
 - 데이터 간단 분석 및 요약통계(describe, mean, std)
 - 데이터 그룹화 및 피벗(pivot), 필터링
 - 데이터프레임 병합(concat, merge)
-

7. 기초 통계학 및 Seaborn 라이브러리

7.1 통계학 기초

- 중심경향성 측정 (평균, 중앙값, 최빈값)
- 데이터 변동성 측정 (분산, 표준편차, 사분위범위)
- 상관관계 및 공분산 분석
- 상관관계와 인과관계의 차이점 및 한계
- 선형회귀 기초 및 최소제곱법(MSE) 개념

7.2 Seaborn 라이브러리를 활용한 데이터 시각화

- 다양한 시각화 기법 (histplot, boxplot, violinplot 등)
 - 범주형 데이터 시각화 (barplot)
 - 산점도 및 회귀선 시각화(regplot, scatterplot)
 - 페어 플롯(pairplot)을 통한 다변수 관계 시각화
 - 상관관계 히트맵(heatmap) 생성 및 분석
 - Anscombe's Quartet 예시를 통한 시각화 중요성 강조
-

8. 선형 회귀 분석 심화

- 단순 선형 회귀(Simple Linear Regression)의 수학적 배경
 - 다중 선형 회귀(Multiple Linear Regression) 소개 및 행렬 표현
 - 정규 방정식(Normal Equation)을 통한 해석적 해법 도출
 - 최적화 문제의 일반적 접근법: 경사 하강법(Gradient Descent) 소개
 - 배치, 미니배치, 확률적 경사하강법의 개념과 차이점
 - 해석적 방법과 경사 하강법 비교 및 활용 예시
 - 회귀 모델 계수 및 잔차(residual) 해석 방법
 - 모델 성능 평가 방법 및 과적합(Overfitting) vs 과소적합(Underfitting) 문제 소개
-

9. 중간고사 안내

- 일시 및 장소
- 시험 범위 및 형식

▼ OT 및 강의 소개

데이터사이언스

- 과학적 방법을 사용하여 데이터에서 정보나 지식을 추출하는 분야임
- 데이터사이언스의 세 가지 구성 요소:
 - 통계학(Statistics)
 - 컴퓨터 과학(Computer Science)
 - 도메인 지식(Domain Knowledge)
- 데이터사이언스는 학제 간 연구 분야임
- 데이터 비즈니스는 데이터를 활용하여 가치를 창출하고 수익을 발생시키며, 디지털 시대 기업 경쟁력의 핵심 요소임
 - 예: 고객 니즈 기반의 콘텐츠 큐레이션

존 스노우의 콜레라 연구 (1854)

- 데이터사이언스의 기념비적 연구 사례 중 하나 (실제 문제를 해결함)
- 당시 사람들은 콜레라가 공기로 전염된다고 생각함
- 존 스노우는 사망 위치와 같은 과학적 데이터를 수집
- 브로드 가(Broad Street)의 물 펌프 주변에서 사망이 집중된 것을 발견함
- 이 연구는 현재의 역학조사의 기초가 되었음

점점 중요해지는 데이터의 역할

- 존 스노우 시대에는 데이터를 손으로 직접 기록해야 했음
- 현재는 저렴하고 접근 가능한 메모리 덕분에 모든 곳에 센서가 있음
- 스마트폰 사진 한 장: 3~5MB (메타데이터 포함 시 더 커짐)
- 스마트폰으로 찍은 30분짜리 영상: 약 12GB
- 이러한 데이터를 편의성, 의사결정, 마케팅 등에 활용 가능함
 - 아마존의 고객 구매 데이터를 통한 고객 니즈 분석
 - 월드컵 2022 축구공 추적

디지털 데이터 이해

- 정보 장치에서 데이터를 표현하기 위해 비트, 바이트 및 더 큰 용량의 단위를 사용함
 - 비트(bit, b)
 - 바이트(byte, B)
 - 킬로바이트(KB)
 - 메가바이트(MB)
 - 기가바이트(GB)
 - 테라바이트(TB)

- 페타바이트(PB)
- 엑사바이트(EB)
- 추가 단위: 제타바이트, 요타바이트, 브론토바이트 등
- ChatGPT: Common Crawl과 같은 대규모 데이터셋(~570GB)에서 훈련됨
 - 이는 약 57만 권의 책을 읽는 것과 비슷함

블랙홀 탐사

- 2019년 5월 10일, EHT 연구팀이 최초로 블랙홀의 그림자를 직접 촬영한 이미지를 공개함
- 단일 망원경으로 관측하기에는 너무 멀리 있었음
- 지구 전체를 하나의 거대한 망원경으로 사용하는 방법을 사용함
 - 총 5페타바이트의 데이터를 수집
 - NumPy를 사용한 엄청난 데이터 처리 후 블랙홀 그림자 이미지 재구성 (상관관계 분석, 교정, 전처리, 이미지 처리 등)

디지털 데이터와 디지털 전환

- 제3차 산업 혁명은 컴퓨팅의 부상으로 시작됨
- 데이터가 빠르게 디지털화되고, 매일 상상하기 어려운 양의 정보를 생성, 저장, 편집, 배포 및 재처리함

디지털 데이터와 디지털 전환

- 데이터: 현실 세계에서 측정된 사실 또는 값
- 정보: 특정 목적을 위해 처리하거나 조직된 데이터
 - 예) 매 1.5미터마다 수집된 온도 데이터: 데이터
 - 시간에 따른 평균 온도, 지역별 차이 계산: 정보

디지털 데이터와 디지털 전환

- 아날로그에서 디지털 데이터로의 변화
- 음성(공기의 진동)
- 아날로그 신호 → 표본화(Sampling), 양자화(Quantization), 디지털화(Digitalization) 과정을 거쳐 디지털 신호로 변환함
 - 표본화(Sampling)
 - 양자화(Quantization)
 - 디지털화(Digitalization)

디지털 데이터와 디지털 전환

- 예시: 비트맵 이미지(회색조 이미지)

새로운 과학 패러다임

- 패러다임: 특정 시대나 분야에서 널리 받아들여진 이론과 관점의 틀임
- 1차 패러다임: 경험적 과학(관찰과 실험)
- 2차 패러다임: 이론적 과학(수학적 모델로 자연 현상 설명)
- 3차 패러다임: 계산 과학(컴퓨터 시뮬레이션으로 현상 모델링 및 예측)
- 4차 패러다임: 데이터 중심 과학(대규모 데이터셋의 수집 및 분석을 통해 새로운 통찰 발견)

배울 내용

- 데이터사이언스의 필수 수학적 기반 (선형대수 및 통계학 포함)
- 데이터 분석을 위한 도구와 라이브러리 소개 (NumPy, Pandas, Matplotlib, Scikit-learn)
- 데이터 수집부터 분석 및 시각화까지 데이터사이언스 프로세스

질문 있습니까?

데이터 분석 개발 도구와 라이브러리

강력한 프로그래밍 언어 (Python)

- Python의 기원: Guido van Rossum이 1989년 개발; 신화 속 뱀 "파이썬"에서 이름을 따옴
- 특징:
 - 효율적 데이터 처리를 위한 내장 자료 구조(리스트, 튜플, 사전, 집합)
 - 인덱싱 및 슬라이싱: 리스트/튜플의 일부 요소를 쉽게 접근하거나 추출 가능
 - 객체 지향 언어: 모든 것이 객체로 취급됨(함수, 모듈 등); 클래스는 메서드를 가짐 (예: 리스트의 sort, append 등)

강력한 프로그래밍 언어 (Python)

- Python은 데이터를 효율적으로 처리하기 위한 내장 자료 구조(리스트, 튜플, 사전, 집합)를 제공함
- Python 리스트와 튜플은 전통적 인덱싱 접근 방식을 통해 데이터를 접근하고, 슬라이싱을 통해 데이터를 쉽게 추출 가능

강력한 프로그래밍 언어 (Python)

- Python은 객체 지향 언어임: 모든 요소(자료형, 함수, 모듈 등)는 객체로 취급됨
 - 리스트 클래스는 sort(), append(), remove(), reverse(), pop()과 같은 내장 메서드를 제공함
-

모듈: 개념 및 사용법

- 모듈: 함수, 클래스, 변수가 포함된 Python 파일(.py)임
- 모듈 사용 이유:
 - 매번 코드를 다시 정의하지 않기 위함 (예: max2()를 max2.py에 저장)
 - 코드 재사용으로 생산성 증가

모듈 활용 및 패키지 설치

- 모듈의 유형:
 - 내장 모듈: Python과 함께 제공됨 (예: math)
 - 사용자 정의 모듈: 사용자가 직접 생성함 (예: my_module.py)
 - 외부 모듈: 제3자가 개발한 것으로 별도 설치가 필요함
- Python에는 100개 이상의 내장 모듈이 제공됨

모듈 활용 및 패키지 설치

- 임포트 방식:
 - import module 또는 from module import func 또는 from module import *
 - 자주 쓰는 모듈에 짧은 이름 지정 가능: import module as md
- NumPy나 Pandas 같은 라이브러리는 Python에 내장되어 있지 않음 (외부 패키지임)
- 외부 패키지는 pip install (일부 OS에서는 pip3 install)을 사용하여 설치 가능

강력한 Python 패키지

- PyPI: Python 패키지 인덱스, 약 300,000개 이상의 프로젝트 (지속적으로 증가)
- 데이터사이언스 라이브러리:
 - NumPy: 배열 수치 연산
 - Pandas: 데이터프레임을 통한 데이터 조작
- 머신러닝 라이브러리:
 - Scikit-learn, TensorFlow 등
- 시각화 라이브러리:
 - Matplotlib, Seaborn

데이터사이언스 라이브러리

- NumPy와 Pandas

데이터사이언스 라이브러리

- 머신러닝 라이브러리

데이터사이언스 라이브러리

- 데이터 시각화 및 그래픽 라이브러리

Anaconda: 설치 및 사용법

- Anaconda: 데이터사이언스 패키지(NumPy, Pandas, TensorFlow, scikit-learn 등)를 포함한 Python 배포판임
- 설치 방법:
 - anaconda.com에서 다운로드
 - TensorFlow 2.x 호환성을 위해 Python 3.6 이상 버전 선택
- Anaconda Navigator:
 - GUI를 통해 JupyterLab, Jupyter Notebook 등의 도구를 실행 가능
 - 본 수업의 주된 개발 환경은 Jupyter Notebook임

Jupyter Notebook 아키텍처

- 브라우저 → 명령 전송 → 노트북 서버 → 커널이 코드 실행 → 결과가 서버를 거쳐 다시 브라우저에 표시됨

Jupyter Notebook 특징

- 웹 기반 인터랙티브 개발 환경:
 - 코드 실행 즉시 피드백을 받을 수 있음
 - 여러 프로그래밍 언어 지원(Python, R, Julia 등)

노트북 셀과 코드 입력

- 셀(Cell):
 - Python 코드를 입력하고 Run 버튼 또는 Shift+Enter로 실행
 - 출력 결과는 셀 아래에 표시됨
- 실행 순서:
 - In [1], In [2] 등으로 표시됨

- 변수는 셀 간에 유지되며, 실행 순서가 중요함
- 저장:
 - 기본적으로 Untitled.ipynb로 자동 저장됨 → 파일명을 클릭하여 변경 가능

노트북 셀과 코드 입력 (계속)

- 동일한 셀을 여러 번 실행하면 이전 값에 계속 누적될 수 있음

Google Colab을 활용한 클라우드 개발

- Google Colab:
 - Jupyter와 유사한 브라우저 기반 환경으로, Google 서버에서 실행됨
 - 데이터/ML 패키지가 사전 설치되어 있음
 - 로컬 설정 없이 바로 사용 가능(colab.research.google.com 접속 후 로그인 필요)

Google Colab 인터페이스

- Jupyter Notebook GUI와 매우 유사함

질문 있습니까?

Anaconda 설치 및 환경 설정

- 가상 환경(Virtual Environment):
 - 독립적인 프로젝트를 위한 전용 공간임
 - 각 프로젝트별로 종속성과 라이브러리를 별도로 관리할 수 있게 도와줌
 - 같은 시스템에서 여러 Python 프로젝트가 종속성 충돌 없이 공존할 수 있게 해줌

가상 환경 생성 및 사용법

- Windows의 경우 Anaconda Prompt가 자동 설치됨
- Mac의 경우 터미널 사용

개인적으로 선호하는 IDE 소개

- VSCode (Visual Studio Code)

Colab 소개

- 노트북 최초 실행 및 런타임 연결
- Markdown 셀과 코드 셀 사용법
- 코드 실행 방법(Shift + Enter, ctrl(cmd) + Enter)
- 코드 셀 실행 중지 방법(ctrl(cmd) + M I)
- 자동 완성 및 문서화 제공 방법
- 오류 메시지 형식화
- Google Drive 연결(mount) 방법

▼ NumPy 라이브러리

Python 리스트 vs. NumPy

- NumPy:
 - 파이썬에서 데이터 과학을 위한 핵심 라이브러리

- 2020년 『Nature』 리뷰 기사에 소개됨
- 모든 요소가 동일한 데이터 타입을 갖는 다차원 배열(텐서)을 다룸
- 데이터 요소의 크기가 균일하므로 빠른 임의 접근(random access)이 가능함
- **Python 리스트:**
 - 다양한 데이터 타입을 저장할 수 있는 유연성은 있지만, 수치 연산에는 느림
- **사용법:**
 - 일반적으로 `import numpy as np` 형태로 임포트
 - 배열 생성 시 `np.array(<list>)` 사용

Python 리스트 vs. NumPy (계속)

- **축 (Axes / 차원):**
 - 1차원 배열 예시: `(3,)`
 - 2차원 배열 예시: `(2,3)`
 - 3차원 배열 예시: `(4,3,2)`
- **요소 (Elements):**
 - 정수 인덱스를 통해 접근
 - `shape (4,3,2)` 는 배열이 3개의 축을 가지며 각 축의 길이가 4, 3, 2임을 의미함

Python 리스트 vs. NumPy (계속)

- **속도:**
 - NumPy는 하나의 데이터 타입만 저장하는 `ndarray` 타입을 사용 → 빠른 접근 및 수학 연산 가능
 - 리스트는 여러 타입을 혼합할 수 있어 접근 속도가 느림
- **벡터 연산:**
 - 리스트는 `+` 연산 시 단순 이어붙이기(concatenation)
 - NumPy 배열은 벡터로 처리되어 요소별 덧셈(element-wise addition)이 수행됨

배열의 주요 속성

- `shape` : 각 차원의 크기를 나타냄 (예: `(2,3)`)
- `dtype` : 데이터 타입을 나타냄 (예: `float64`)
- `size` : 전체 요소 수
- `ndim` : 차원의 수 (1차원, 2차원 등)

배열의 주요 속성 (계속)

- 산술 연산은 요소별로 수행되며, 두 배열 간의 연산은 인덱스를 기준으로 요소가 대응됨

브로드캐스팅 (Broadcasting)

- 브로드캐스팅은 작은 배열이나 단일 값을 더 큰 배열과 연산할 때 확장시켜 연산 가능하게 함
- 벡터화 연산(vectorized operation)은 하나의 명령으로 여러 요소에 동시에 적용 가능
- 예: 배열에 `10` 을 곱하면 모든 요소에 각각 적용됨

브로드캐스팅 예시

특수 배열 생성

- `zeros((n, m))` : $n \times m$ 크기의 0으로 채워진 배열 생성
- `ones((n, m))` : $n \times m$ 크기의 1로 채워진 배열 생성
- `full((n, m), x)` : $n \times m$ 크기의 배열을 값 x 로 채움
- `eye(n)` : $n \times n$ 크기의 단위 행렬 생성

연속적인 수열 생성

- `arange(m, n, step)` : Python의 `range` 와 유사하며, n 미만의 값을 `step` 간격으로 생성
- `linspace(start, stop, num)` : 시작값과 종료값 사이를 동일한 간격으로 `num` 개 생성 (종료값 포함)
- `logspace` : 로그 간격으로 수열을 생성함

연속적인 수열 생성 (계속)

배열 차원 및 삽입

- `(3,)` : 1차원 배열
- `(1,3)` 또는 `(3,1)` : 2차원 배열
- `insert(a, index, value, axis)` : 지정 위치에 새 값이나 행/열을 삽입
- `flip(a, axis)` : 지정한 축을 기준으로 배열 요소를 뒤집음
- 삽입 시 자동으로 브로드캐스트가 발생할 수 있음

인덱싱, 슬라이싱, 논리 인덱싱

- 2차원 배열의 인덱싱은 일반적으로 `[행, 열]` 사용
- 슬라이싱은 각 차원에서 일부분을 선택할 수 있음
- 슬라이싱 예시는 `:` 와 `,` 의 사용에 주의
- 논리 인덱싱은 조건문(예: 나머지 연산 등)을 사용하여 해당 조건을 만족하는 요소만 선택
- 예: 2의 배수인 요소만 선택
- 논리 인덱싱 시각화 가능

최대값, 최소값, 평균, 정렬

- `max`, `min`, `mean` 등의 메서드는 배열 값을 요약
- `astype` 은 데이터 타입 변환 가능 (예: `float64` → `int`)
- `flatten` : 다차원 배열을 1차원으로 펼침
- `T` : 행과 열 전치(transpose)
- `sort(axis=n)` : 지정 축 기준으로 행 또는 열을 오름차순 정렬

배열 추가 및 재구성

- `append` : 지정한 축을 따라 두 배열을 결합하거나, 축이 없으면 평평하게 병합

- `reshape(new_shape)` : 데이터 수를 유지하면서 배열의 차원 형태만 변경

난수 생성 (의사 난수)

- 컴퓨터는 실제 난수가 아닌 '의사 난수(pseudo-random)'를 생성
- 시드(seed)를 설정하면 동일한 결과 재현 가능 → 실험이나 시뮬레이션에 유용

NumPy의 난수 모듈

- `randint` : 정수 난수 생성
- `randn`, `normal` : 정규 분포를 따르는 실수 난수 생성
- 시드를 고정하면 항상 같은 난수 결과를 얻을 수 있음

NumPy가 성공한 이유 (배열 프로그래밍과 벡터화)

- 배열 프로그래밍은 개별 요소가 아닌 데이터 전체를 한 번에 처리할 수 있게 함
- 벡터화는 데이터 수준 병렬처리로, 하나의 명령이 다수의 요소에 병렬로 적용됨
- 비벡터화: 각 요소를 순차적으로 처리
- 벡터화: 모든 요소를 병렬로 처리 (하드웨어가 병렬 연산 지원 시 성능 향상)

벡터화 성능 측정

- 예: 처리 시간이 약 20배 단축됨
- 요소 수가 100,000개인 배열의 경우 → 약 60배 이상 단축 가능

축소 연산 (Reduction Operations)

- 배열의 여러 값을 하나의 값으로 축약 (예: `sum`, `mean`)
- 이러한 연산도 벡터화되어 수동 반복보다 훨씬 빠름

선형대수 (연립방정식 풀이)

- `numpy.linalg` 모듈 내의 `solve` : 연립방정식 해 구하기
- `det` : 행렬식 계산

배열 결합: `concatenate`, `vstack`, `hstack`

- `concatenate` : 지정 축을 따라 배열을 결합
- `vstack` : 수직 방향으로 배열 결합
- `hstack` : 수평 방향으로 배열 결합
- 대형 배열 생성 시 자주 사용됨

▼ 최적화 문제

갑자기 최적화 알고리즘을 다루는 이유

데이터 사이언스는 '의사결정 최적화'와 깊은 관련이 있음

데이터 사이언스의 핵심 목표는 단순히 데이터를 분석하는 것이 아니라,

| 데이터 기반으로 더 나은 결정을 내리는 것

이를 위해서는 다음과 같은 수학적 문제를 푸는 능력이 필요

- 자원은 한정되어 있고(시간, 비용, 칼로리 등),
- 선택 가능한 옵션이 여럿 있으며,
- 어떤 조합이 **가장 효과적**인지 판단해야 함.

👉 이때 바로 **최적화 모델**이 등장

성능, 정확성, 계산 자원 사이의 **트레이드오프**를 이해하는 것은 데이터 과학자에게 매우 중요한 사고 방식

계산 모델과 최적화의 연관성 강조

강의 초반에 나왔던 키워드들을 연결해 보면:

- **Computational model** → 세상을 수학적으로 추상화
- **Optimization model** → 가장 좋은 결정을 찾는 수단
- **Simulation model** → 시나리오를 실험해보기

즉, 데이터 사이언스는 단순 통계 분석이 아니라 **계산 모델링 + 최적화 + 예측**을 조합하여 실제 문제를 해결하는 학문

최적화 문제

계산 모델 (Computational model)

- 컴퓨테이션(계산)을 사용하여 세상을 이해하고자 함
- 현실에서 무슨 일이 일어났는지 이해하거나 미래를 예측하는 실험 장치로 사용
- 실험이 점차 습식 실험실(wet labs)에서 컴퓨터로 옮겨가고 있음
- 주요 모델들:
 - 최적화 모델 (Optimization models)
 - 통계 모델 (Statistical models)
 - 시뮬레이션 모델 (Simulation models)

최적화 모델 (Optimization model)

- ****목적함수(Objective function)****로 시작하며, 이는 **최대화 또는 최소화**되어야 함
- 예: 최소 시간 경로 (전체 이동 시간을 최소화)
- 그다음 ****제약 조건(constraints)****이 존재함
- 예: 비용 제약 조건 ("하지만 저렴해야 해요...")

배낭 문제 (Knapsack problem)

- 가장 높은 가치를 가지는 항목들을 선택하고자 함
- **운반 가능한 무게의 한계**가 있음
- 어떤 물건을 선택하고, 어떤 것을 포기할지 어떻게 결정할까?
- 두 가지 주요 변형:

1. 연속형/분수형 배낭 문제 (Continuous, fractional knapsack problem)

→ 물건(예: 금)을 일부만 가져갈 수 있음

2. 0/1 배낭 문제 (0/1 knapsack problem)

→ 물건을 가져가거나 가져가지 않거나 둘 중 하나만 가능

- 0/1 문제는 더 어려움

→ 이전 선택이 이후 선택에 영향을 미치기 때문

0/1 배낭 문제 (정의)

- 각 항목은 <가치, 무게> 의 쌍으로 표현됨
- 배낭은 총 무게가 w 이하인 항목들만 담을 수 있음
- 길이 n 의 벡터 I : 사용 가능한 항목들의 리스트
- 길이 n 의 벡터 V : 각 항목을 선택했는지 여부 (0 또는 1)
- $V[i] = 1$ 이면 항목 $I[i]$ 는 선택됨
- 최적화 문제 정식화:
- 목적함수:

$\sum_{i=0}^{n-1} V[i] \times I[i].value$ 를 최대화

- 제약조건:

$\sum_{i=0}^{n-1} V[i] \times I[i].weight \leq w$

무차별 대입(brute force) 알고리즘

1. 가능한 모든 항목 조합(power set)을 나열
2. 허용 무게를 초과하는 조합은 제거
3. 나머지 조합 중 가치가 가장 높은 조합 선택

무차별 대입 알고리즘의 실용성

- power set(멱집합)은 얼마나 큰가?
- V 는 n 비트로 표현 가능한 모든 이진수만큼의 조합을 가짐

→ 가능한 V 의 개수 = 멱집합의 수

- 예: 선택할 수 있는 항목이 100개라면?

→ 멱집합의 수: $2^{100} \approx 2^n$

0/1 배낭 문제 예시

- 제한: 1500 칼로리
- 알고 있는 정보:
- 각 항목의 가치
- 각 항목의 칼로리
- 목표: 칼로리 제한을 지키면서 가치를 최대화

탐욕 알고리즘 (Greedy algorithm)

- 미리 정해진 탐욕적(greedy) 기준에 따라 항목을 선택
- 단, 0/1 배낭 문제에서는 탐욕 알고리즘이 최적 해를 보장하지 않음

탐욕 알고리즘의 장단점

장점:

- 구현이 쉬움
- 계산 효율이 높음

단점:

- 항상 최적 해를 제공하지 않음
 - 얼마나 좋은 해인지, 근사 정도가 얼마나 우수한지 판단할 방법이 없음
-

최적화 문제 (Optimization problems)

0/1 배낭 문제

- 제약 조건이 있는 **정형화된 최적화 문제**
 - 데이터 사이언스의 핵심 개념과 밀접한 연관:
 - 최적화 (Optimization)
 - 자원 제약 (Resource constraints)
 - 계산 복잡도 (Computational complexity)
 - 알고리즘 기반 문제 해결 (Algorithmic problem-solving)
-

0/1 배낭 문제 예시

예시 상황:

칼로리 제한: 1500

알고 있는 것:

- 각 항목(음식)의 **개인적 가치(value)**
- 각 항목의 **칼로리(calorie)**

목표:

주어진 칼로리 제한 내에서 **가치를 최대화**

탐욕 알고리즘의 장단점

장점:

- 구현이 간단함
- 계산 효율이 높음

단점:

- 항상 최적의 해를 제공하지 않음
 - 근사 해의 질이 얼마나 좋은지 알기 어려움
-

브루트포스(무차별 대입) 알고리즘

1. 모든 항목 조합(**power set**)을 나열
 2. 총 무게(또는 칼로리)가 허용 범위를 초과하는 조합은 제거
 3. 남은 조합 중 **가치가 가장 높은 조합 선택**
-

브루트포스 알고리즘의 실용성

- 멱집합(power set)은 얼마나 큰가?
- V 는 n 비트로 표현할 수 있는 모든 이진수 조합만큼 존재함
- 즉, 가능한 V 의 수 = 멱집합의 수
- 예시: 선택할 항목이 100개라면?

→ 멱집합 수: $2^{100} \approx 2^n$

이진 탐색 트리 (Binary Search Tree)를 활용한 구현

구현 방법

- 항목 리스트에서 첫 번째 항목을 선택
- 배낭에 공간이 있다면(예: 더 많은 칼로리를 먹을 수 있다면)
→ 왼쪽 자식 노드에 삽입
- 공간이 없다면
→ 오른쪽 노드에 삽입
- 단, 리프 노드를 제외한 모든 자식 노드에 대해 재귀적으로 탐색 수행
 - 제약을 만족하면서 가치가 가장 높은 노드를 찾는 것이 목표

예시 (이진 탐색 트리 기반)

음식	맥주(beer)	피자(pizza)	햄버거(burger)
가치	90	30	50
칼로리	154	258	354

이진 탐색 트리의 시간 복잡도

- 시간 복잡도는 트리 내 노드 수에 비례
- 트리의 레벨(깊이) = 항목 수
- i 번째 레벨의 노드 수 = 2^i
- 따라서 전체 노드 수 = 2^{n+1}
- 시간 복잡도: $O(2^{n+1})$
 - 최적화 여지:
 - 제약을 미리 위반한 가지(branch)는 탐색 생략
 - 전체 시간 복잡도 자체는 같지만, 실제 실행 시간은 단축 가능

이진 탐색 트리의 실제 구현

- 명시적으로 트리를 구축하지 않음
- 대신, **지역 변수 `result` **를 사용하여 현재까지의 최적 해를 기록

실습 자료 예시

- 항목 수가 2^8 개 정도면 어떤 알고리즘도 처리 가능

- 문제가 커지면(더 큰 메뉴) 어떻게 될까?

시뮬레이션: 큰 메뉴 구성

- `np.random` 을 활용하여 가치와 칼로리를 무작위로 할당
- 대규모 데이터셋을 구성해 실험 수행

동적 프로그래밍 (Dynamic Programming: 메모이제이션 기반)

피보나치 수열 예시

- 재귀적 방법
- 동일한 부분 문제를 반복해서 계산

피보나치 수열 – 메모이제이션 적용

- 이전에 계산한 결과가 있다면 저장하여 재사용(memoization)
- `fib(x)` 를 계산하기 전에 이미 존재하는지 확인
- 있으면 그대로 사용
- 없으면 계산 후 테이블에 저장

동적 프로그래밍이 가능한 조건

1. 최적 부분 구조 (Optimal substructure)
 - 전체 최적 해는 부분 문제의 최적 해를 조합해 만들 수 있음
 - 예: `fib(x) = fib(x-1) + fib(x-2)`
2. 중복 부분 문제 (Overlapping subproblems)
 - 동일한 부분 문제를 여러 번 해결해야 함

0/1 배낭 문제에 동적 계획법(DP) 적용 가능?

- 최적 부분 구조 존재? → 예
 - 중복 부분 문제 존재? → 예
- 따라서 동적 계획법 적용 가능

동적 프로그래밍을 활용한 0/1 배낭 문제

- 각 노드의 상태:
- {선택된 항목들}, [남은 항목들], 누적 가치, 사용 가능한 칼로리

핵심 정리 (Takeaways)

- 많은 현실 세계의 문제들은 최적화 문제로 공식화할 수 있음
- 목적 함수(objective function)를 최소화/최대화하면서 제약 조건을 만족
- 0/1 배낭 문제의 경우:
 - 탐욕 알고리즘은 간단하고 효율적이며 적절한 해를 제공하지만, 항상 최적은 아님

- **동적 프로그래밍**은 조건이 맞는 경우 정확하고 좋은 성능을 보여줌
(최적 부분 구조 + 중복 부분 문제 존재 시)

▼ 데이터 시각화

데이터 사이언스에서 시각화의 역할

- 대형 언어 모델(LLM)의 부상 예시: **표(table)** vs ****산점도(scatter plot)****로 표현 방식 비교
- 데이터의 구조를 시각적으로 파악하게 해줌

선 그래프 (Line Chart)

- 시간에 따른 변화나 추세를 나타내기 위해 사용됨
- 연속형 데이터에 적합함
- 여러 개의 **시계열을 비교**할 수 있어, 함께 상승/하강하거나 갈라지는 경향을 시각적으로 확인 가능
- 데이터가 논리적 순서를 따름을 나타냄 (예: 일일 기온)
- **최고점, 최저점, 점진적 변화** 등을 부드럽게 관찰할 수 있음

파이 차트와 막대 그래프 (Pie chart & Bar chart)

파이 차트

- 하나의 변수에 대한 **범주별 비율**을 원형으로 표현
- 전체에 대한 각 범주의 **기여도를 직관적으로 전달**함

막대 그래프

- 범주별 **값(빈도, 평균, 합계 등)**의 차이를 비교
- 각 범주 간 **수량적 차이**를 명확하게 보여줌

히트맵 (Heatmap)

- 이미지 위에 **데이터의 분포를 열(heat)** 형태로 표현함
- 예: 위성사진에 표시된 실제 열 분포 지도
 - 일반적으로는 **서로 다른 값을 서로 다른 색상**으로 나타냄 → 데이터 분포 파악에 유리
 - 다양한 ****컬러맵(color map)****을 사용할 수 있음

산점도 (Scatter Plot)

- 두 변수 간의 **상관관계**를 ****직교 좌표계(Cartesian Plane)****에 점을 찍어 표현
- 점의 **기울기**가 관계의 방향을 보여줌
 - Positive: 양의 상관관계
 - Negative: 음의 상관관계
 - **점들이 밀집**될수록 관계가 강함
 - 주된 분포에서 **멀리 떨어진 점들은 이상치(outliers)**
 - 관계는 선형, 곡선, 무작위 등 다양할 수 있음

Q: 산점도와 선 그래프의 차이?

→ 산점도는 데이터 간의 관계, 선 그래프는 시간 또는 순서상의 변화를 시각화

히스토그램 (Histogram)

- 막대 그래프의 일종
 - 연속형 데이터를 구간(bin)으로 나누고, 각 구간의 빈도를 막대로 표현
 - bin 수가 많을수록 세밀한 분포, 적을수록 거친 분포
 - 누적 히스토그램은 각 구간 이하의 누적 데이터 수를 보여줌
-

상자 수염 그래프 (Box and Whisker Plot)

- 중앙값, 사분위수, 이상치를 시각적으로 요약
 - 상자: 제1사분위수(Q1) ~ 제3사분위수(Q3) 포함
 - 중앙값: 상자 내부에 선으로 표시됨
 - 수염(Whisker): 상자 밖으로 퍼진 데이터 범위 표현
 - 이상치: 수염 바깥의 점으로 표현
- 데이터의 분포, 치우침(skew), 극단값 판단에 유용
-

시각화 퀴즈

1. 회사 예산에서 다섯 부서가 차지하는 비율을 한눈에 보여주고 싶을 때
→ 정답: A. 파이 차트
 2. 200명이 좋아하는 과일(사과, 바나나, 오렌지, 키위)에 대한 설문 결과를 비교할 때
→ 정답: B. 막대 그래프
 3. 한 달 간의 아이스크림 매출과 일별 기온 데이터를 통해 상관관계를 분석할 때
→ 정답: B. 산점도
 4. 4가지 비료 방식으로 재배한 토마토의 무게 분포를 비교하고 싶을 때
→ 정답: C. 상자 수염 그래프
-

요약

어떤 시각화 기법을 선택할지는 다음에 따라 달라짐:

- 보유하고 있는 데이터의 종류
 - 시각화를 통해 표현하고 싶은 정보
-

Python의 matplotlib 을 활용한 데이터 시각화

Matplotlib: 시각화 도구

- 파이썬에서 가장 인기 있는 시각화 라이브러리
- 막대 그래프, 선 그래프, 파이 차트, 산점도, 상자-수염 그래프 등을 그릴 때 사용됨
- `pyplot` 서브 모듈을 주로 사용
-

plt.plot

- `plt.plot(x, y)` 는 `x` 와 `y` 리스트(또는 배열)를 입력으로 받음
- `plt.xlabel()` , `plt.ylabel()` 은 x축과 y축에 **축 이름(label)**을 작성함
- `plt.show()` 는 그래프를 출력함 (단, 주피터 노트북에서는 생략해도 자동 출력됨)
- `plt.axis()` 는 x축과 y축의 **표시 범위**를 설정함

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(10)          # 0부터 9까지 정수 배열 생성
plt.plot(x**2)             # x^2 함수를 선 그래프로 그림
plt.axis([0, 100, 0, 100]) # x축: 0~100, y축: 0~100 범위로 설정
```

plt.plot 의 선 및 마커 모양 커스터마이징

- `plt.plot` 그래프에서 **선 스타일과 마커 모양**을 설정할 수 있음

예시:

- `'g--'`: 녹색 점선
- `'r^--'`: 빨간 선 + 삼각형 마커
- `'b*.'`: 파란 점선 + 별표 마커
- 여러 개의 함수를 하나의 그래프에 함께 그릴 수도 있음

```
import matplotlib.pyplot as plt
import numpy as np

x = np.arange(-20, 20)
y1 = 2 * x
y2 = (1/3) * x ** 2 + 5
y3 = -x ** 2 - 5

plt.plot(x, y1, 'g--', x, y2, 'r^--', x, y3, 'b*:')
plt.axis([-30, 30, -30, 30])
plt.show()
```

연습: 한 그래프에 세 개의 함수 그리기

함수:

- $y = 2x$
- $y = \frac{1}{3}x^2 + 5$
- $y = -x^2 - 5$

연습 문제 (Exercise)

아래 그림처럼 한 그래프에 사인(sine) 함수와 코사인(cosine) 함수를 함께 그려보세요.

제목, 축 라벨, 범례 설정

- 제목: `plt.title('제목')`
- 축 이름: `plt.xlabel()` , `plt.ylabel()`
- 범례: `plt.legend()`

Figure와 Axes 구조 (상세 설명)

- **Figure**: 그림 전체를 감싸는 틀 (여러 개의 그래프를 포함할 수 있음)
- **Axes**: 실제 그래프가 그려지는 **좌표 공간**
- `plt.subplots()` → 하나의 Figure에 여러 개의 그래프를 동시에 배치 가능

```
f, ax = plt.subplots(nrows=2, ncols=2, figsize=(8, 6))  
# 2행 2열, 전체 크기는 8x6인치
```

그래프 그리기 함수 예시

```
fig, ax = plt.subplots(2, 2)  
  
# (0, 0) 산점도  
X = np.random.randn(100)  
Y = np.random.randn(100)  
ax[0, 0].scatter(X, Y)  
  
# (0, 1) 막대 그래프  
X = np.arange(10)  
Y = np.random.uniform(1, 10, 10)  
ax[0, 1].bar(X, Y)  
  
# (1, 0) 선 그래프 (코사인)  
X = np.linspace(0, 10, 100)  
Y = np.cos(X)  
ax[1, 0].plot(X, Y)  
  
# (1, 1) 히트맵  
Z = np.random.uniform(0, 1, (5, 5))  
ax[1, 1].imshow(Z)
```

아이리스 데이터셋 (The Iris dataset)

Iris 데이터셋 시각화 실습

다음 세 가지 방법으로 Iris 데이터를 시각화해 봅시다:

1. 히스토그램 (Histogram)
2. 산점도 (Scatter Plot)
3. 상자 수염 그래프 (Box Plot)

▼ Pandas 라이브러리

Pandas

- 파이썬의 기본 자료형인 리스트(List)와 딕셔너리(Dictionary)는 유연하지만 **표 형식(tabular)** 데이터를 다루기에는 최적화되어 있지 않음
- "패널 데이터(panel data)"에서 유래되었으며, **R 언어의 데이터프레임(DataFrame)** 구조에서 강한 영향을 받음
- 일반적으로 **Anaconda 배포판**에 기본 포함되어 있으나, 없을 경우 다음과 같이 설치 가능:

`pip install pandas` 또는 `conda install pandas`

Pandas vs. Excel

- Excel**은 스프레드시트 기반의 데이터 작업에 널리 사용되는 프로그램
- 데이터 과학자들은 일반적으로 **2차원 행렬 형태의 구조화된 데이터(tabular data)**를 선호함 → 행/열 단위 접근이 편리하기 때문
- Pandas**는 **NumPy** 기반으로 빠른 처리 속도와 유연한 구조화 데이터 처리 기능 제공

왜 Pandas를 사용하는가?

- NumPy**를 기반으로 한 고성능 처리
- 행과 열 조작을 위한 **DataFrame** 구조 제공
- 데이터 정제, 분석, 변환을 위한 **다양한 내장 함수** 지원

Pandas는 어떤 작업에 사용되는가?

- 세기, 처리, 결측치(missing data) 필터링
- 표 형식 데이터의 **삽입 및 삭제**
- 열 데이터 정렬 등 데이터 조작**
- 필드 간 상관관계 분석**
- 파이썬의 리스트, 딕셔너리, NumPy 배열을 **간편하게 DataFrame으로 변환**
- matplotlib**과의 연동을 통해 시각화 가능

Pandas의 두 가지 핵심 자료 구조

1. Series

- 1차원 레이블 배열**
- 스프레드시트의 **하나의 열(column)** 과 유사
- **인덱스(index)****와 ****값(values)****으로 구성됨
- 파이썬 리스트처럼 인덱싱 가능하지만, **더 유연한 라벨 사용 가능**

2. DataFrame

- Pandas의 **2차원 레이블 데이터 구조의 핵심**
- 행과 열로 구성

- 각 열은 서로 다른 데이터 타입을 가질 수 있음
- 월별 수입, 지출 등 표 형태의 데이터 표현에 적합

Pandas가 NumPy 2D 배열보다 뛰어난 이유

- NumPy는 레이블(행/열 이름)을 저장하지 않음
- Pandas는 레이블을 유지하므로 데이터의 맥락 이해가 쉬움
- NumPy 배열은 모든 요소가 동일한 데이터 타입이어야 함
- Pandas는 여러 데이터 타입을 혼합하여 저장 가능

DataFrame 생성하기

- **Series나 리스트로 구성된 딕셔너리(dictionary)**로부터 생성 가능

DataFrame 생성 예시

- Jupyter Notebook에서는 DataFrame을 표(table) 형식으로 자동 출력함

→ 행: index, 열: columns

외부 파일에서 DataFrame 읽기

- CSV(.csv), TSV(.tsv), Excel(.xlsx) 등 다양한 외부 파일로부터 불러올 수 있음
- CSV: 콤마(,)로 구분된 값
- TSV: 탭(\t)으로 구분된 값

```
pd.read_csv('path_to_csv_file')
df_excel = pd.read_excel('data.xlsx', sheet_name='Sheet1')
```

- index_col=0 옵션을 사용하면 특정 열을 인덱스로 지정할 수 있음

결측치(Missing Data)

- 실제 데이터에는 종종 정제되지 않았거나 누락된 값이 존재
- Pandas는 이를 탐지하고 처리할 수 있는 함수를 제공함
- isna(): 결측치를 검사하여 True/False 결과 반환

DataFrame 시각화

- Pandas는 데이터를 빠르게 시각화할 수 있는 기능을 제공
- matplotlib 과 결합하여 막대 그래프, 선 그래프 등 다양한 시각화 가능

Pandas: Series, DataFrame

- Pandas는 두 가지 강력한 데이터 구조를 도입함:
- **Series**: 1차원 레이블 배열. 스프레드시트의 열(column) 과 유사하며, **인덱스(index)**와 **값(values)**으로 구성됨
- **DataFrame**: 서로 다른 타입의 열들을 가질 수 있는 2차원 레이블 구조

→ Python에서 테이블 또는 스프레드시트처럼 생각할 수 있음

슬라이싱과 인덱싱 (Slicing and Indexing)

- 처음 또는 마지막 몇 개의 행을 조회할 때는 `head()` 와 `tail()` 을 사용
- 매개변수 없이 사용하면 기본적으로 **처음 5개와 마지막 5개 행**을 보여줌

- 리스트 슬라이싱처럼 특정 범위의 행을 조회할 수 있음

→ 예: 인덱스 2부터 5까지 조회하면 **2, 3, 4, 5번째 행**이 출력됨

- 특정 ****행 인덱스(label)****를 이용하여 조회할 때는 `df.loc[]` 사용

→ 여러 행을 선택하려면 인덱스 배열을 전달하면 됨

- 열 선택 시에는 ****열 이름(column index)****으로 조회 가능

- **행과 열 슬라이싱/인덱싱을 결합**할 수도 있음

→ 예: `'2011'` 열에서 **0번째와 4번째 행**을 선택

→ 또는 `loc` 을 사용해 행과 열의 레이블을 직접 지정

! 주의: `loc['Korea', '2011']` 형태 사용. 리스트 이중 중첩 아님!

- `head()` , `tail()` 과 인덱싱을 함께 사용할 수 있음

`loc` 와 `iloc` 을 사용한 슬라이싱과 인덱싱

- `df.loc[]` : ****인덱스 레이블(label)****을 사용
- `df.iloc[]` : ****정수 인덱스(integer index)****를 사용

→ 모두 `[행, 열]` 형식으로 사용할 수 있음

- 예시: `df.loc['Korea':'US', '2009':'2011']`

→ 특정 행과 열 범위 슬라이싱 가능

Boolean 인덱싱을 사용한 선택

- 조건식을 통해 Boolean 인덱싱 가능

예: `df[df['2007'] > 10]`

→ 2007년도 값이 10 초과인 행만 선택

열 추가하기

- DataFrame에 새 열을 추가하는 방법:

```
df['New Column'] = some_values
```

열 삭제하기

- `df.drop()` 을 사용하여 열 삭제

- `inplace=True` 파라미터 사용 시, 원본에서 즉시 반영됨

```
df.drop('ColumnName', axis=1, inplace=True)
```

Pandas를 이용한 간단한 데이터 분석

- 기상 데이터(.csv)를 Pandas로 분석
- Github에서 직접 csv 불러오기

→ 한글이 포함되어 있으므로 `encoding='utf-8-sig'` 지정 필요

1단계: `describe()` 로 기본 통계 정보 확인

- 데이터 수(count), 평균(mean), 표준편차(std), 최소/최대 값 등을 확인할 수 있음
- 문자열 열은 수치 연산이 불가능하므로 분석에서 제외됨
- 특정 통계만 확인하려면 `mean()`, `std()` 등 개별 함수 사용 가능

표준편차 주의사항

- Pandas는 **표본 표준편차(sample std)** 사용
- NumPy는 **모집단 표준편차(population std)** 사용

→ 계산식이 다름에 주의

결측치 처리

- `count()` 를 통해 **각 열의 값 개수**를 확인할 수 있음

→ 열마다 개수가 다른 이유는 **결측치가 존재하기 때문**

- `isna()` 함수로 결측치 여부 확인 가능

→ NaN인 위치는 `True` 로 표시되는 Boolean DataFrame 반환

```
df.isna()
```

- Boolean 인덱싱으로 결측치가 있는 행만 선택 가능

결측치 삭제 또는 채우기

- `dropna()` : 결측치가 있는 행 또는 열 삭제
- `fillna()` : 결측치를 특정 값으로 채움

시계열 분석을 위한 DatetimeIndex

- **DatetimeIndex**를 사용하면 날짜 데이터를 연/월/일/시/분 등으로 쉽게 파싱할 수 있음

→ 시계열 데이터 분석에 매우 유용

연습 1

- DatetimeIndex를 활용하여 ****월별 평균 풍속(line graph)****을 그려보세요

→ 힌트: 'month' 열 추가 → Boolean 인덱싱 → for loop 사용

데이터 그룹화 및 필터링

- 값에 따라 데이터를 그룹핑하거나 필터링 가능

→ Excel의 필터 기능처럼 사용

→ `groupby()` 메서드로 그룹화 수행

연습 2

- 연습 1과 동일한 결과를 만들되, **for loop 없이** 구현하세요

Boolean 인덱싱을 활용한 필터링

- 조건식을 기반으로 원하는 데이터 필터링 가능

→ 예: `df[df['month'] == 6]`

`pivot()` 을 이용한 데이터 구조 변경

- 새로운 인덱스와 열 구조를 지정하여 DataFrame을 재구성
- `pivot()` 결과에는 결측치가 생길 수 있으므로 `fillna()` 로 보완

`concat()` 을 이용한 DataFrame 병합

- 두 개 이상의 DataFrame을 세로 또는 가로 방향으로 합칠 수 있음

```
pd.concat([df1, df2], axis=0) # 행 방향 병합
pd.concat([df1, df2], axis=1) # 열 방향 병합
```

→ 수직 또는 수평 방향으로 간단히 병합

`concat()` 을 이용한 병합 – 추가 설명

- `pd.concat()` 은 두 DataFrame을 세로 방향(**axis=0**) 또는 가로 방향(**axis=1**) 으로 쌓아 합침
- 열(column)이 일치하지 않는 경우를 다루기 위해 **join** 키워드 파라미터 사용 가능

```
pd.concat([df1, df2], axis=0, join='outer') # 기본값은 outer join
```

`merge()` 와 `concat()` 의 차이

`pd.concat()`

- 단순히 위 또는 옆으로 이어붙이는 방식
- SQL 스타일의 조인과는 다름
- `axis=0` : 행 기준으로 병합 (기본값)
- `axis=1` : 열 기준으로 병합
- `join` 파라미터로 열이 일치하지 않을 때 처리 방식 설정
- `'outer'` : 누락된 열은 NaN으로 채움

- 'inner': 공통된 열만 유지

pd.merge() 또는 df.merge()

- SQL의 JOIN 연산처럼, 키 값 기준으로 병합
- on='column_name': 병합 기준이 되는 키 열 지정
- how='inner' | 'left' | 'right' | 'outer': 조인 방식 지정
 - 'inner': 공통된 키만 병합
 - 'left': 왼쪽 DataFrame 기준
 - 'right': 오른쪽 DataFrame 기준
 - 'outer': 전체 키 포함 (없는 값은 NaN)

```
pd.merge(left_df, right_df, on='column', how='inner')
# 또는
left_df.merge(right_df, on='column', how='inner')
```

merge() 의 동작 방식

- merge() 는 기존 인덱스를 버리고, 새로운 숫자 인덱스를 생성함
- on 파라미터: 병합 기준이 되는 키 열 지정
- how 파라미터: 병합 방식을 선택

요약하자면,

함수	기능 요약	주요 사용 시기
concat()	단순히 DataFrame을 위 또는 옆으로 이어붙임	인덱스 또는 열을 기준으로 정렬 없이 붙이고 싶을 때
merge()	키 열을 기준으로 SQL JOIN처럼 병합	공통 열(키)을 기준으로 정렬 및 병합이 필요할 때

▼ 통계 및 Seaborn 라이브러리 그리고 선형 회귀

통계 및 Seaborn 라이브러리

데이터 과학을 위한 통계

- 데이터 과학은 데이터를 수집, 요약, 해석하는 활동을 포함함
- 분포와 요약 통계량의 이해는 탐색과 모델링의 기초임
- 고급 기법들도 기초 통계 직관(예: 상관관계 vs 인과관계)에 의존함

이미 학습한 통계 지표 예:

→ 평균(mean), 최소값(min), 최대값(max), 표준편차(standard deviation)

중심 경향성 척도 (Measures of Central Tendency)

- 평균(Mean): 값들의 평균
 - 이상치에 민감함
 - 대칭적이고 단봉 분포(unimodal)에 적합

- **중앙값(Median):** 정렬된 값의 가운데
 - n이 홀수 → 중앙 값
 - n이 짝수 → 중간 두 값의 평균
 - 이상치에 강건함 → **왜도 있는 분포에 적합**
- **최빈값(Mode):** 가장 자주 나타나는 값
 - 범주형 데이터나 이산형 변수에 유용

변동성 척도 (Measures of Variability)

- **분산(Variance):** 평균으로부터의 제곱 거리의 평균
- **표준편차(Standard Deviation):** 원래 데이터와 같은 단위를 가짐 → 해석 용이
- **사분위 범위(IQR):**
 - 75백분위수(Q3) - 25백분위수(Q1)
 - 표준편차보다 **이상치에 더 강건함**

상관관계와 공분산

- **공분산(Covariance):** 두 변수가 함께 혹은 반대로 움직이는지
 - 함께 증가하면 양수, 반대로 움직이면 음수
- **상관계수(Pearson's r):**
 - 공분산을 정규화한 지표 (범위: -1 ~ +1)
 - +1: 강한 양의 선형 관계
 - 1: 강한 음의 선형 관계
 - 0: 선형 관계 없음 (비선형일 수 있음)

상관관계 vs 인과관계

- **상관관계:** 두 변수가 함께 움직이는 것
- **인과관계:** 하나의 변수가 다른 변수에 **직접적인 영향을 줌**

예: 아이스크림 판매량과 익사 사고 → 상관관계는 있지만 인과관계는 아님

→ 실제 원인은 "기온 상승" → 더 많은 수영 → 익사 증가

주의:

- 높은 상관관계 ≠ 인과
- 비선형 관계일 경우, 상관계수는 0에 가까울 수 있음

NumPy를 이용한 상관분석

```
import numpy as np

x = np.arange(0, 10)
y1 = x * 2 + 1
np.corrcoef(x, y1)

# 무작위 예제
```

```
np.random.seed(2)
y2 = np.random.randint(0, 100, size=10)
np.corrcoef(x, y2)
```

- 두 개 이상의 배열을 비교하려면 **튜플(tuple)** 형태로 전달

단순 선형 회귀 (Simple Linear Regression)

- 선형 회귀는 **x와 y 간의 선형 관계**를 가정함
- 식:

$$\hat{y}(x) = \beta_0 + \beta_1 \hat{y}(x) = \beta_0 + \beta_1 x$$

- β_0 : 절편, β_1 : 기울기
- 목표는 $\hat{y}$ 가 실제 y에 최대한 가깝게 되도록 하는 것

→ 이를 위해 ****비용 함수(cost function)****를 최소화함

예: 평균제곱오차(MSE)

선형 회귀는 머신러닝이다

- 어떤 함수(선형, 다항, 기타)를 **추정**함
- **loss 함수(오차 함수)** 기준으로 **적합도**를 평가
- 수집된 **입력-출력 데이터**를 기반으로
 - 이는 **지도학습(supervised learning)**의 한 형태

Seaborn 라이브러리

- **Matplotlib** 기반의 고수준 시각화 라이브러리
- `matplotlib` 과 함께 사용할 수 있음
- 내장된 예제 데이터 제공 (예: `tips`, `iris` 등)

주요 Seaborn 시각화 함수

분포 시각화

- `sns.histplot(df['sales'], bins=10)` : 히스토그램
- `sns.boxplot()` : 중앙값, 사분위수, 이상치 시각화
- `sns.violinplot()` : 분포 + 밀도 표현

범주형 데이터 시각화

- `sns.barplot(x='category', y='sales', data=df)`

산점도 및 회귀선

- `sns.scatterplot(x='sales', y='profit', data=df)`
- `sns.regplot(x='sales', y='profit', data=df)` → 회귀선 포함

Pairplot

- `sns.pairplot(df, vars=['sales', 'profit'], hue='category')`
 - 여러 수치형 열 간의 관계를 한 번에 파악
 - 히스토그램 + 산점도 자동 생성

상관 행렬 히트맵 (Heatmap)

```
corr = df.corr()
sns.heatmap(corr, annot=True, fmt='.2f', cmap='Blues')
```

→ 변수 간 상관계수를 시각적으로 확인 가능

Seaborn과 `tips` 데이터셋 시연

- 레스토랑 계산서 및 팁 데이터셋

→ Seaborn에 내장된 DataFrame 형태

산점도 예시

```
# matplotlib 사용 시
plt.scatter(df['total_bill'], df['tip'])

# seaborn 사용 시
sns.scatterplot(x='total_bill', y='tip', data=df)
```

→ 팁은 총계(total bill)가 커질수록 증가하는 경향

`regplot()` 으로 회귀선 추가

```
sns.regplot(x='total_bill', y='tip', data=df)
```

히스토그램 그리기

```
sns.histplot(df['tip'])
```

→ 보통 팁은 2~4달러 사이에 분포

Pairplot으로 변수 간 관계 확인

- 수치형 변수 간 **쌍별 관계 시각화**
- 예: `total_bill` 과 `tip` 은 상관관계가 높음
- 반면 `group_size` 는 이산형 → 점이 계단식으로 나타남

Anscombe's Quartet 데이터셋

- 동일한 평균, 분산, 상관계수를 갖는 4개의 데이터셋
- 시각화해보면 전혀 다르게 보임
- 시각화의 중요성과 이상치의 영향 강조

선형 회귀 - 다시 정리

- 목표:

$$\hat{y}(x) = \beta_0 + \beta_1 x$$

→ 예측값 $\hat{y}$ 와 실제값 y 의 오차를 최소화

- 비용 함수: MSE (Mean Squared Error)

→ 편미분을 통해 β_0, β_1 을 구함

▼ 선형 회귀2

단순 선형 회귀 (Simple Linear Regression)

- 선형 회귀는 **파라미터에 대한 비용 함수(cost function)**를 최소화함으로써 수행됨

- 대표적인 비용 함수 예시: **평균제곱오차 (MSE, Mean Squared Error)**

→ 목표:

$$\hat{y}(x) = \beta_0 + \beta_1 x$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}(x_i))^2 \quad \text{MSE} = \frac{1}{n} \sum_{i=1}^n \left(y_i - \hat{y}(x_i) \right)^2$$

→ β_0, β_1 값을 찾아 이 오차를 최소화하는 것이 목적

단순 선형 회귀 - 수학적 접근

- 단순 선형 회귀에서는 MSE를 **편미분하여 0으로 만드는** 방식으로 최적 파라미터를 구함

→ 즉, 미분 후 **극값 조건**을 만족하는 β_0, β_1 을 찾음

다중 선형 회귀 (Multiple Linear Regression)

목표

- 여러 개의 **독립 변수(x 벡터)**를 입력으로 받아
- **종속 변수(y)**를 예측하는 시스템을 구축하는 것

수식 표현

$$\hat{y} = X\beta$$

- β : 학습할 파라미터 벡터
 - 목적: MSE 손실 함수를 최소화하는 β 찾기
-

행렬 형태로 표현

$$\text{디자인 행렬 } X \in \mathbb{R}^{n \times (d+1)}$$

- 각 행은 하나의 데이터 포인트를 의미
- 보통 첫 번째 열은 **1로 고정** (절편(intercept) 항 포함)

$$\text{파라미터 벡터 } \beta \in \mathbb{R}^{(d+1)}$$

- 절편 β_0 과 각 변수에 대한 기울기 $\beta_1, \beta_2, \dots, \beta_d$ 포함

예측 벡터 $\hat{y} \in \mathbb{R}^n$

- X 의 각 행과 β 를 내적하여 계산된 예측 결과

실제 타겟 벡터 $y \in \mathbb{R}^n$

- 각 데이터 포인트의 실제 정답값을 담은 벡터

다중 선형 회귀에서의 해 구하기

- 가중치 벡터 β 를 어떻게 구할까?

β

- 수식 구조는 비교적 단순
- 비용 함수의 극값을 직접 구할 수 있음

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

→ 단, $X^T X$ 가 가역 행렬(invertible)이어야 위 식을 사용할 수 있음

→ 이 **정해진 해(closed-form solution)**로 손실 함수(MSE)를 최소화하는 최적 β 를 구할 수 있음

선형 회귀는 머신러닝이다

- 우리는 어떤 함수를 근사하고 있음
 - 선형일 수도 있고, 다항식이나 비선형일 수도 있음
 - **적합도 점수(goodness-of-fit)**에 따라 평가
 - 그 기준이 바로 **손실 함수(loss function)**
 - 다수의 **입력-출력 데이터 포인트**를 기반으로 함
- 이것이 바로 머신러닝(Machine Learning)
- 특히 **지도학습(Supervised Learning)**의 대표적인 형태

만약 비용 함수가 이런 형태라면?

→ 좀 더 일반화된 **수학적 최적화(Mathematical Optimization)** 방법이 필요함

예:

- 경사하강법(Gradient Descent)
- 정규화(Regularization) 포함
- 비선형/다항 회귀 등 다양한 함수 형태에 대한 일반 해법 적용

▼ 경사하강법

경사하강법 (Gradient Descent)

경사하강법이란?

- 비용 함수(cost function)를 최소화하기 위한 **일반적인 수치 최적화 방법**
- 함수가 복잡하거나, 해를 직접 구하기 어려울 때 유용

경사하강법 예제 1

우리는 다음 함수를 최소화하고자 함:

$$y = (x-3)^2$$

1. 어느 방향으로 이동할까? (기울기의 반대 방향)
2. 얼마나 이동할까? (학습률 η 사용)

경사하강법 예제 2

$$y = (x_1 - 2)^2 + 3(x_2 + 1)^2$$

1. 이동 방향은? → 기울기의 반대 방향
2. 이동량은? → 기울기 크기 $\times$ 학습률

실습 예제: 선형 회귀의 경사하강법

- 선형 회귀 문제에서의 **MSE 손실 함수**를 기억하자
 - 손실 함수는 **가중치 β** 에 대한 함수임
- 우리는 $y = f(x)$ 가 아닌
- **$loss = L(\beta)$** 를 최소화하려는 것

배치 경사하강법 (Batch Gradient Descent)

데이터가 10,000개일 때, 경사하강법의 방식은 다음과 같음:

1. **배치 경사하강법**: 모든 데이터를 이용하여 손실값과 기울기를 계산 후, **한 번에 업데이트**
2. **확률적 경사하강법 (SGD)**: 한 번에 **한 개 데이터**만 사용하여 업데이트
3. **미니 배치 경사하강법**: 소수의 데이터를 묶어서 계산 및 업데이트

선형 회귀: 해석적 해 vs 경사하강법

해석적 해 (Analytic Solution)

- 정규 방정식 사용:
$$\hat{\beta} = (X^T X)^{-1} X^T y$$
- 전제조건: $X^T X$ 가역(invertible)해야 함
- 계산은 빠르지만 **대규모 데이터**에서는 메모리 문제 발생 가능

경사하강법 해

- 손실 함수의 기울기(**gradient**)를 반복적으로 계산하여 업데이트
- 반복 공식:
$$\beta := \beta - \eta \cdot \nabla L(\beta)$$
 - η : 학습률(learning rate)
- **반복(iterative)** 방식 → 역행렬 불필요 → **확장성 좋음**

비교 요약

항목	해석적 해 (정규방정식)	경사하강법 (GD)
----	---------------	------------

수식	단한 해 Closed-form	반복적 Iterative
속도 (소규모 데이터)	빠름	느림
속도 (대규모 데이터)	느리고 메모리 소모 큼	배치로 효율적
역행렬 필요 여부	필요	불필요
일반화 가능성	선형 모델에만 가능	딥러닝 등 다양한 모델에 사용 가능

예시: 특성(feature)이 1,000개, 데이터가 1천만 개라면? → 경사하강법이 적합

회귀 계수 해석 (Interpretation of Coefficients)

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_d x_{id} \quad \hat{y}^{(i)} = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_d x_{id}$$

- β_0 : 절편 → 모든 특성이 0일 때의 예측값
- β_j : x_{ij} 가 예측값 y 에 미치는 **한계 영향**
- 부호(+/-): 영향 방향을 의미

예시: 혈압(mmHg)을 예측하는 모델에서

x_{i2} = 나이(연도), $\beta_2 = 1.5$ 라면?

→ 나이가 1세 증가할 때, 혈압이 평균적으로 **1.5mmHg 증가**

잔차 해석 (Residuals)

$$\text{Residual}(i) = y(i) - \hat{y}(i) = y_i - \hat{y}_i \quad \text{Residual}^{(i)} = y^{(i)} - \hat{y}^{(i)} = y_i - \hat{y}_i$$

- 잔차: 각 데이터 포인트에서의 **예측 오차**
- 잔차가 0에 가까울수록 → 예측이 정확
- 잔차는 다음 문제를 진단하는 데 도움:
 - 비선형성 (misspecification)
 - 이분산성 (heteroscedasticity)
 - 이상치 (outliers)

학습 성능을 어떻게 정의할까?

- 학습 데이터만으로 평가하는 것은 불공정
→ 학습된 파라미터가 본 데이터만 잘 맞도록 과적합될 수 있음

→ 따라서 일반적으로 ****성능 측정용 데이터셋(test set)****을 따로 마련함

용도	설명
train set	모델 학습에 사용
test set	성능 측정에 사용 (처음 보는 데이터)

과소적합 vs 과적합 (Underfitting vs Overfitting)

용어	정의	예시
과소적합	모델이 너무 단순하여 데이터 패턴을 잡지 못함	곡선 데이터를 직선으로 피팅
과적합	모델이 너무 복잡하여 노이즈까지 학습함	고차 다항식

적절한 피팅	일반화 성능이 좋은 모델	적절한 차수의 다항식
--------	---------------	-------------

예시: 실제로는 이차 함수 + 잡음

- 좌측부터:

1. 직선 모델 → 과소적합
2. 이차 모델 → 적절
3. 9차 다항식 → 과적합

일반화 오류 정의

- **훈련 오차 (Training Error):** 학습에 사용된 데이터에서의 예측 오차
- **테스트 오차 (Test Error):** 학습하지 않은 데이터에서의 예측 오차
→ 일반화 성능을 평가하는 핵심 지표